UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

Information and Communication Technology Department

FINAL THESIS REPORT

Tran Hien Chuong

Student ID: 22BA13056

Supervised by:

Dr. Tran Hoang Tung

Title:

AI Assistant for Vietnamese Labor Law: A Case Study on Employment Contract Termination

Program: Information and Communication Technology

Academic Year: 2025–2026

SUPERVISOR CERTIFICATION AND SIGNATURE PAGE

I certify that this thesis report was prepared by Tran Hien Chuong under my supervision. The work presents the design, implementation, deployment architecture, and evaluation of an AI/software system for citation-grounded Vietnamese labor-law question answering.

Supervisor signature: Date:

Dr. Tran Hoang Tung

ACKNOWLEDGEMENT

I would like to thank my supervisor, Dr. Tran Hoang Tung, for his guidance throughout this project. I am also grateful to the instructors and classmates at the University of Science and Technology of Hanoi for comments on the system design, evaluation methodology, and report structure. This work depends on open-source software and public engineering documentation from the Python, FastAPI, Qdrant, Neo4j, Next.js, and retrieval-augmented generation communities.

Contents

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
BLLD	Vietnamese Labor Code 2019
BLTTDS	Vietnamese Civil Procedure Code 2015
E2E	End-to-End
LLM	Large Language Model
QA	Question Answering
RAG	Retrieval-Augmented Generation
RRF	Reciprocal Rank Fusion
SBERT	Sentence-BERT

List of Tables

List of Figures

ABSTRACT

This thesis, *AI Assistant for Vietnamese Labor Law: A Case Study on Employment Contract Termination*, presents a citation-grounded AI assistant for Vietnamese labor-law question answering. The work is framed as a software system thesis rather than a pure model-comparison study. The system addresses a practical limitation of flat and hybrid retrieval-augmented generation: legal answers require preservation of statutory hierarchy, exact citations, cross-references, and bounded refusal when the indexed corpus does not contain the required legal instrument.

The implemented system processes a scoped six-document Vietnamese labor-law corpus into hierarchy-aware chunks, enriches each chunk with legal metadata and citation surfaces, indexes dense and sparse retrieval signals in Qdrant, expands retrieved evidence through a Neo4j legal graph, validates generated citations deterministically, and exposes the assistant through a FastAPI backend and Next.js frontend. The deployment architecture separates a Vercel-hosted frontend from a Render-hosted backend, with Qdrant and Neo4j used as retrieval stores.

The official evaluation uses a deterministic 100-query benchmark with 94 in-corpus questions and 6 out-of-corpus refusal tests. On the final retrieval ablation, graph-augmented retrieval improves Recall@10 from 0.735 to 0.835 and retrieval pass rate from 0.649 to 0.766, while Forbidden Citation Violation Rate increases from 0.032 to 0.043. The final end-to-end run obtains an adjusted pass rate of 0.780, citation validation pass rate of 1.000, quality validation pass rate of 0.980, and out-of-corpus QA pass rate of 1.000. The results indicate strong benchmark-level citation containment and refusal behavior, while retrieval robustness for paraphrased, multi-hop, hard-negative, table, and labor-dispute procedure queries remains the main limitation.

Keywords: Vietnamese labor law, legal question answering, retrieval-augmented generation, hybrid retrieval, knowledge graph, citation validation

1 Introduction

1.1 Context and Problem Statement

Legal question answering is useful only when a user can inspect the source of an answer. In Vietnamese labor law, the source is not merely a paragraph with similar wording. A reliable answer must preserve the document, article, clause, point, appendix, and legal rank that support the response. It must also avoid answering when the required legal instrument is outside the indexed corpus. This makes Vietnamese labor-law QA a software system problem as much as a language-model problem.

The implemented project began from practical employment questions such as contract termination, severance, retirement, minor workers, working time, and labor disputes. These topics quickly showed that a narrow set of termination rules is not enough. A question about ending an employment relationship may require the Labor Code, an implementing decree, a ministerial circular, a retirement-age table, or a civil procedure provision. A flat retriever can return semantically similar passages, but it does not naturally preserve this legal hierarchy.

Standard retrieval-augmented generation systems usually split documents into passages, retrieve the most similar passages, and place them into a generator prompt [7]. This approach is fragile for statutory QA. A legal rule may depend on a parent article, a lower-level implementing document, or a cross-reference that is not semantically close to the user query. Dense embeddings may also miss exact article numbers and table references, while generated answers may invent citations unless they are checked after generation.

The core problem addressed in this thesis is therefore how an implemented Vietnamese labor-law assistant can retrieve, assemble, and validate legally grounded evidence while remaining bounded to a scoped corpus. The system is designed around the idea that evidence should remain inspectable from ingestion to final answer: legal units are preserved during processing, represented in Qdrant and Neo4j during retrieval, assembled into citation-bearing contexts, and checked again during answer validation.

1.2 Objectives and Scope

The thesis has four software objectives. First, it builds a reproducible corpus pipeline that turns curated Vietnamese labor-law texts into hierarchy-aware evidence chunks. Second, it implements hybrid dense-sparse retrieval in Qdrant together with policy-controlled graph expansion in Neo4j. Third, it connects retrieval to grounded answer construction, deterministic citation validation, and bounded insufficient-context behavior. Fourth, it evaluates the deployed software behavior on a fixed 100-query benchmark with both in-corpus and out-of-corpus cases.

The official corpus contains six document groups: the Labor Code 2019, the labor-only subset of the Civil Procedure Code 2015, Decree 135/2020/ND-CP, Decree 145/2020/ND-CP, Circular 09/2020/TT-BLDTBXH, and Circular 10/2020/TT-BLDTBXH. The system is a legal information assistant for these indexed sources. It is not a legal advice tool, and it does not claim correctness for laws, amendments, or instruments outside the corpus.

The project does not train a new language model and does not attempt to compare language models as the main research object. Its focus is the implemented software path from legal corpus processing to retrieval, graph expansion, answer construction, citation validation, frontend/backend operation, deployment, and evaluation.

1.3 Contributions and Thesis Organization

The main contributions are concise and implementation-oriented:

- a hierarchy-aware processing pipeline for Vietnamese labor-law provisions;
- a hybrid Qdrant retrieval layer linked to a Neo4j legal graph;
- deterministic citation validation and bounded insufficient-context behavior;
- a FastAPI and Next.js software implementation suitable for deployment;
- a reproducible 100-query benchmark evaluation using official artifacts.

The rest of the thesis follows this system framing. Section 2 reviews legal QA, RAG, hybrid retrieval, knowledge graphs, and the research gap. Section 3 describes the system design and methodology. Section 4 explains implementation and deployment. Section 5 presents evaluation results and error analysis. Section 6 concludes and identifies future work.

2 Background and Related Work

2.1 Legal Question Answering and Citation Grounding

Legal QA differs from open-domain QA because the answer must be authoritative, traceable, and bounded. In this thesis, authority means that a cited provision belongs to the indexed legal corpus and has an identifiable statutory coordinate. Traceability means that each final answer citation can be matched to a retrieved context. Boundedness means that the assistant should refuse or disclose insufficient context for unsupported topics.

Legal NLP benchmarks such as LexGLUE [1] and LegalBench [4] show the importance of legal-domain evaluation, but they do not directly measure Vietnamese labor-law article retrieval with deterministic citation validation. The benchmark in this thesis is smaller and more specialized. It is designed to test whether the implemented system retrieves required legal bases and avoids benchmark-defined forbidden citations.

For Vietnamese labor law, citation grounding is a product requirement rather than a presentation detail. Users need to know whether an answer depends on the Labor Code, an implementing decree, a circular, an appendix, or the labor-related subset of the Civil Procedure Code. A useful assistant must therefore expose evidence and reject unsupported legal instruments instead of relying on model memory.

2.2 Retrieval-Augmented Generation and Hybrid Retrieval

Retrieval-Augmented Generation combines an external retriever with a generator so that generated text can be conditioned on retrieved evidence [7]. The retriever normally embeds a query and searches an indexed document collection. The generator then writes an answer from the retrieved context. Dense retrieval methods such as DPR [6] and sentence embedding methods such as SBERT [8] support semantic matching beyond exact keywords.

For general QA, this design reduces reliance on model memory and makes knowledge updates easier. For legal QA, it is only the starting point. Legal users need exact legal bases, not only relevant passages. The retriever must recover statutory coordinates, tables, and cross-referenced provisions. The generator must

be constrained by retrieved context and checked after generation because hallucinated legal citations are a known risk in neural generation [5].

Hybrid retrieval combines dense semantic search with sparse lexical search. Dense vectors help match concepts expressed in different wording. Sparse lexical signals help match exact legal terms, article numbers, document identifiers, and short coordinate expressions. In Vietnamese labor law, both are required. A user may ask an informal question such as whether a company can transfer a worker to another job for nearly three months, but the legal answer depends on a precise article. Conversely, a user may ask directly about a legal coordinate, in which case exact matching should dominate.

The implemented system uses Qdrant to store dense and sparse vectors together with metadata payloads. The payload is not auxiliary decoration; it is part of the legal contract between retrieval and answer validation. It stores document ID, article number, clause, point, normative rank, citation text, and taxonomy labels.

2.3 Knowledge Graphs for Legal Retrieval

Graph-augmented retrieval uses graph structure to add relational context beyond the seed passages found by search. Microsoft GraphRAG uses graph communities and generated summaries for broad corpus exploration [2]. The graph in this thesis has a different purpose. It is a deterministic provision-level legal graph. Its nodes and edges are built from known document structure, cross-reference extraction, metadata, and normative rank.

This design is appropriate for a scoped legal assistant because graph traversal can recover context that is not semantically close to the user query but is legally connected. Examples include a Labor Code article and its implementing decree, a circular that lists permitted light work for minors, a retirement-age appendix, or a civil procedure article needed for labor-dispute jurisdiction.

2.4 Research Gap

RAG evaluation frameworks such as Ragas [3] and ARES [9] motivate separation between retrieval quality, answer quality, and faithfulness. This thesis adopts that separation but uses deterministic software checks rather than LLM-as-Judge scoring for final metrics. Deterministic checks are narrower than human legal review, but they are repeatable. For this project, repeatability is important because the system is a software artifact whose behavior must be inspected and improved across retrieval, graph expansion, citation validation, and refusal logic.

The research gap is an engineering gap: a complete citation-grounded Vietnamese labor-law assistant over a scoped corpus. The contribution is not a new LLM, nor a claim that graph expansion solves legal reasoning. The contribution is the integration of corpus processing, hybrid retrieval, graph expansion, deterministic validation, API/frontend deployment, and benchmark reporting into a coherent system.

3 System Design and Methodology

3.1 System Requirements and Overview

The system is organized into two pipelines. The offline pipeline prepares legal evidence. It validates curated legal texts, splits them into hierarchy-aware chunks, enriches metadata, extracts references, builds

the Qdrant index, and constructs the Neo4j legal graph. The online pipeline answers user queries. It retrieves seed chunks, expands them through graph relations and fallbacks, reranks and budgets context, constructs an answer, validates citations, and returns either a grounded answer or insufficient-context response.

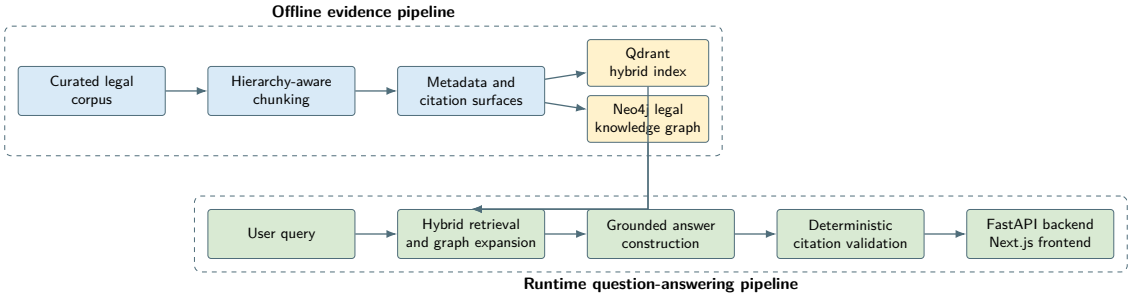

Figure 1. Overall System Architecture.

The design requirements follow directly from the legal QA setting: preserve statutory coordinates, retrieve both semantic and exact-coordinate evidence, use graph relations without allowing uncontrolled expansion, validate all final citations against retrieved context, and refuse questions whose legal basis is outside the indexed corpus.

3.2 Legal Corpus and Hierarchy-Aware Processing

The corpus is intentionally scoped to six legal document groups. This scope supports reproducible evaluation and bounded system claims. Table 1 is based on `artifacts/index/current.json`.

Table 1. Scoped Legal Corpus Summary

Document ID	Role in Corpus	Rank	Chunks
`45-2019-qh14`	Labor Code 2019	1	697
`92-2015-qh13-labor-only`	Civil Procedure Code labor subset	1	159
`nghi-dinh-135-2020-nd-cp`	Retirement age decree	2	41
`nghi-dinh-145-2020-nd-cp`	Labor Code implementation decree	2	520
`thong-tu-09-2020-tt-bldtbxh`	Minor worker circular	3	73
`thong-tu-10-2020-tt-bldtbxh`	Labor contract circular	3	66
Total	Six indexed document groups		1,556

The rank column encodes the legal hierarchy used by the system. Rank 1 contains primary codes, rank 2 contains decrees, and rank 3 contains circulars. This hierarchy does not decide legal correctness by itself, but it helps context assembly avoid treating lower-level guidance as a substitute for primary statutory authority.

The system uses curated text files rather than raw OCR during the final pipeline. The cleaning and validation stage normalizes text, checks required fields, and preserves official document boundaries. The chunking stage is hierarchy-aware: it detects articles, clauses, points, and appendices instead of cutting by arbitrary character windows. Each chunk stores both text for retrieval and text for citation. This

separation matters because search benefits from parent headings and expanded context, while citations need stable human-readable labels.

The key metadata fields include identifiers, document labels, normative rank, article and clause coordinates, citation text, retrieval text, and taxonomy labels for topic, actor, and issue type. These fields are carried through Qdrant payloads and used later by validation.

3.3 Hybrid Retrieval and Legal Knowledge Graph

For a query q, Qdrant returns an initial seed set:

$$S_k = \text{HybridSearch}(q, k).$$

Hybrid search combines dense Vietnamese SBERT vectors with sparse lexical matching. Dense vectors support conceptual similarity. Sparse signals support exact coordinates and legal vocabulary. The final index manifest records `keepitreal/vietnamese-sbert`, 768-dimensional dense vectors, a sparse vector channel, and 1,556 indexed records.

The graph is designed for provision-level expansion. It represents evidence chunks, legal documents, articles, clauses, points, appendices, topics, actors, and issue types. Edges represent structural hierarchy, source links, cross-references, taxonomy membership, and normative hierarchy. The graph is deterministic: it is built from processed artifacts and rules rather than from open-ended LLM extraction.

At runtime, seed chunks are mapped to Neo4j nodes through their shared chunk identifiers. The graph expander then adds related contexts:

$$C_{\text{graph}} = \text{Expand}_G(S_k, d, E),$$

where d is the configured expansion depth and E is the set of allowed edge types. Direct coordinate fallback also retrieves missing explicit article references when possible. The final context set is:

$$C_{\text{final}} = \text{Rerank}(S_k \cup C_{\text{graph}} \cup C_{\text{fallback}}).$$

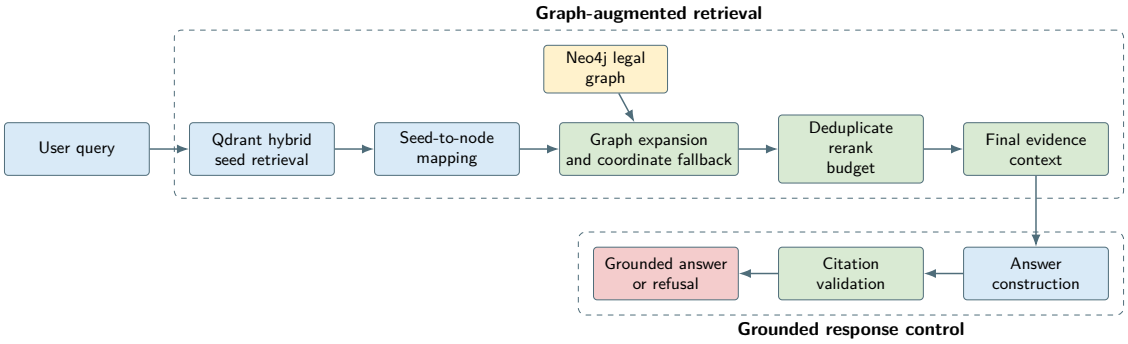

Figure 2. Graph-Augmented Retrieval Flow.

3.4 Answer Construction and Citation Validation

The answer layer is deliberately conservative. It receives only budgeted retrieved contexts. It can use provider-backed generation when configured, but the official final evaluation uses a deterministic extractive provider. The validator checks citation containment: every legal basis cited in the answer must be available in the retrieved context, and article numbers mentioned in the answer must be supported by retrieved metadata. If a generated answer fails validation, the system can return an extractive fallback or insufficient-context response.

3.5 Evaluation Design

The evaluation separates retrieval, answer quality, citation validation, and refusal behavior. Table 2 summarizes the metrics used in Section 5.

Table 2. Evaluation Metrics Summary

Metric	Scope	Meaning
Recall@10	Retrieval	Fraction of required citations recovered in the top-10 retrieved contexts.
Required Citation Coverage	Retrieval	Citation-level coverage over in-corpus queries.
Forbidden Citation Violation Rate	Retrieval	Share of in-corpus queries where a benchmark-defined distracting citation appears.
Retrieval pass rate	Retrieval	Query-level pass requiring required citations and no forbidden violation.
Answer pass rate	Answer	Share of answers passing deterministic answer-presence and rule checks.
Citation validation pass rate	Answer	Share of answers whose citations remain inside retrieved context.
Quality validation pass rate	Answer	Share of answers passing deterministic quality checks.
Adjusted end-to-end pass rate	Full QA	In-corpus QA pass plus out-of-corpus refusal pass over all 100 queries.
Out-of-corpus QA pass rate	Safety	Share of unsupported queries refused or marked insufficient.

4 Implementation and Deployment

4.1 System Modules and Technology Stack

The implementation is organized around repository modules rather than a monolithic script. The `scripts/` directory contains corpus, graph, index, and evaluation commands. The Python package under `src/vn_labor_law_ai_assistant/` contains retrieval, graph, answering, API, auth, conversation, and observability modules. The `frontend/` directory contains the Next.js interface. The `docs/` directory records architecture and reproducibility instructions, and `artifacts/evaluation/final/` stores the official evaluation outputs used in this thesis.

Table 3 summarizes the main modules and technologies.

Table 3. Technology Stack and System Modules

Layer	Technology	System role
Corpus pipeline	Python scripts	Validate, chunk, enrich, and export legal evidence.
Embeddings	Sentence Transformers	Build dense Vietnamese SBERT vectors.
Vector search	Qdrant	Dense and sparse hybrid retrieval with payload metadata.
Graph store	Neo4j	Legal hierarchy, references, taxonomy, and expansion.
Backend API	FastAPI	Chat, health, auth, admin, and conversation routes.
Frontend	Next.js	Chat interface, admin pages, and backend proxy.
Evaluation	Python scripts	Retrieval ablation, end-to-end metrics, and failure reports.
Deployment	Vercel and Render	Frontend and backend hosting architecture.

4.2 Offline Indexing and Graph Loading

The offline pipeline follows the reproducibility guide. The main scripts are:

- `validate_curated_legal_texts.py` for curated source checks;
- `build_legal_chunks.py` for hierarchy-aware chunking;
- `enrich_legal_chunks.py` for metadata and citation surfaces;
- `build_reference_edges.py` for cross-reference extraction;
- `build_index.py` for Qdrant hybrid indexing;
- `build_legal_graph.py` for Neo4j graph construction.

The official index manifest records 1,556 chunks from six documents, no duplicate chunk IDs, no missing retrieval text, no missing citation text, preserved document types, preserved normative ranks, and a successful validation status. The vector store uses Qdrant payload records, dense vector name `dense`, sparse vector name `sparse`, and collection name `vietnamese_labor_law_chunks`.

4.3 Runtime QA Pipeline

The retrieval modules perform hybrid search, payload filtering, graph expansion, fallback, scoring, and context assembly. The runtime formula is:

$$C_{\text{final}} = \text{Rerank}(S_k \cup C_{\text{graph}} \cup C_{\text{fallback}}).$$

The implementation uses chunk IDs as the bridge between Qdrant and Neo4j. This avoids fuzzy linking at runtime. A context entering the answer layer has both text and metadata, which lets the citation validator inspect legal coordinates rather than only raw strings.

The answer layer is implemented in the `rag/answering` modules. The key functions parse answer payloads, format legal bases, build extractive fallbacks, and validate grounded answers. The validation rules check whether cited legal bases are present in the retrieved contexts and whether mentioned article numbers are supported. This design treats citation validation as part of the product behavior, not just as an evaluation metric.

The official final evaluation uses the deterministic extractive provider. This provider is less fluent than a carefully constrained LLM, but it is reproducible and exposes retrieval failures clearly. When evidence is missing, no generator can recover the correct legal basis safely.

4.4 Deployment and Reproducibility

The backend is a FastAPI application with routes for health checks, chat, authentication, conversations, and admin functions. The frontend is a Next.js application with chat and admin pages. The frontend can proxy chat requests to the backend through a configured `BACKEND_URL`. This separation makes it possible to review the UI independently and then connect it to the hosted backend.

The deployment architecture separates browser UI, backend orchestration, and retrieval stores. The Next.js frontend is designed for Vercel hosting. The FastAPI backend can run on Render with environment variables for Qdrant, Neo4j, provider settings, CORS origins, and authentication. Qdrant stores the hybrid index, while Neo4j stores graph relations used during expansion.

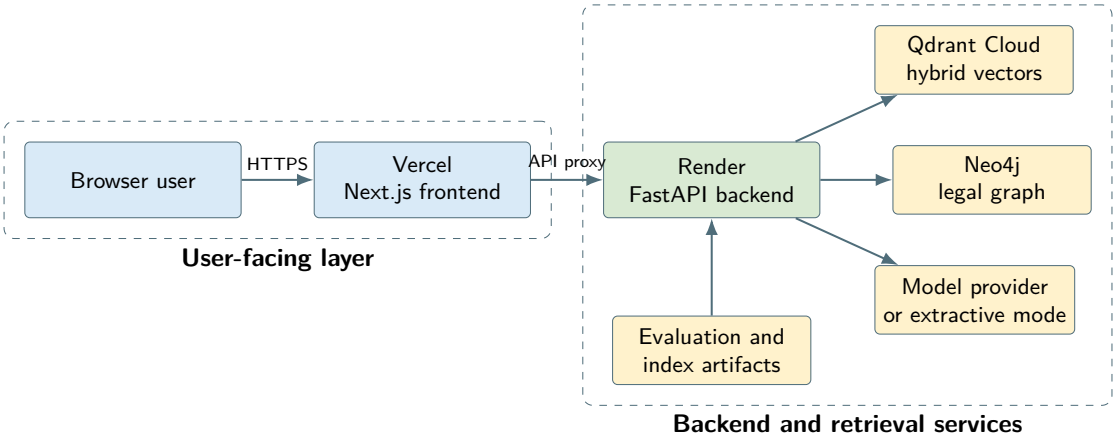

Figure 3. Deployment Architecture with Vercel frontend and Render backend.

Reproducibility is supported by fixed commands, saved JSON/CSV/Markdown reports, and deterministic evaluation scripts. The official final run uses the 100-query benchmark, top-k 10, prefetch limit 24, maximum answer contexts 8, provider `extractive`, graph expansion on 89 queries, and average graph depth 2.160. The final reports under `artifacts/evaluation/final/` are treated as the source of truth for Section 5.

5 Evaluation and Discussion

5.1 Experimental Setup

The final evaluation is deterministic and artifact-based. Its sources are the final retrieval ablation report, the final end-to-end report, the 100-query benchmark file, and the current index manifest stored under `artifacts/evaluation/final/`, `artifacts/evaluation/`, and `artifacts/index/`.

The benchmark has 100 queries: 94 in-corpus queries and 6 out-of-corpus refusal tests. Retrieval metrics are computed on the 94 in-corpus queries because out-of-corpus queries have no required citation set. End-to-end metrics are adjusted across all 100 queries: in-corpus queries must pass retrieval, answer, citation, and quality checks; out-of-corpus queries must refuse or report insufficient context.

Table 4 summarizes the benchmark categories.

Table 4. Benchmark Composition

Category	Queries
Direct QA	19
Definition QA	7
Document guidance QA	10
Exception-based QA	11
Multi-hop QA	7
Table and appendix lookup	10
Hard-negative citation QA	5
Paraphrased real-user QA	5
Labor dispute and procedure QA	5
Procedure QA	7
Comparison QA	4
Scenario-based QA	4
Out-of-corpus QA	6
Total	100

5.2 Retrieval and Graph Expansion Results

Table 5 compares hybrid-only retrieval with graph-augmented retrieval. Graph expansion improves Recall@10 and Required Citation Coverage from 0.735 to 0.835. It also improves retrieval pass rate from 0.649 to 0.766. The trade-off is a small increase in Forbidden Citation Violation Rate from 0.032 to 0.043.

Table 5. Retrieval and Graph Expansion Results

Mode	Queries	Recall@10	Coverage	Forbidden rate	Pass rate
Hybrid-only	94	0.735	0.735	0.032	0.649
Graph-augmented	94	**0.835**	**0.835**	0.043	**0.766**

The result supports the main architectural choice. Graph expansion helps recover related legal provisions that hybrid retrieval alone misses. However, graph expansion is not automatically safe. A nearby legal provision can be related but still be a forbidden citation for the benchmark query. This explains the small increase in forbidden citation violations.

5.3 End-to-End QA and Citation Validation Results

Table 6 reports the final end-to-end run. The adjusted pass rate is 0.780. The citation validation pass rate is 1.000, with no unsupported article numbers and no unretrieved citations. This shows that the citation guard is effective on the benchmark. The lower end-to-end score is caused mainly by retrieval failures.

Table 6. End-to-End QA and Citation Validation Results

Metric	Value
Total benchmark queries	100
Adjusted end-to-end pass rate	0.780
Retrieval pass rate	0.780
Answer pass rate	0.980
Citation validation pass rate	**1.000**
Quality validation pass rate	0.980
Average final quality score	94.11
Low-information quotes	1
Unsupported article numbers	**0**
Unretrieved citations	**0**
Out-of-corpus QA pass rate	**1.000**
Graph expansion used	89 queries
Average graph depth	2.160

5.4 Case Studies and Error Taxonomy

Table 7 combines representative case studies with the main error categories reported by the official final evaluation.

Table 7. Case Studies and Error Taxonomy

Case or error	Result	Interpretation
Female retirement age in 2026	**Pass**	Retrieved Labor Code and Decree 135 table evidence.
Minimum wage by region in 2026	**Pass refusal**	Correctly identified insufficient indexed context.
Temporary transfer paraphrase	Fail	Missed Labor Code Article 29 required citations.
Domestic worker contract template	Fail	Missed appendix/form evidence and produced low-information answer.
Holiday overtime pay hard negative	Fail	Mixed missing citation with over-expansion to overtime limit provisions.
Labor-dispute court choice	Fail	Cross-code procedure retrieval remained incomplete.
Retrieval missing required context	20 cases	Dominant failure reason.
Retrieval over-expansion	4 cases	Related but forbidden citations entered top results.
Low-information answer	2 cases	Extractive answer lacked enough useful rule content.
Answer missing required rule	1 case	Answer quality failed after retrieval gap.

5.5 Discussion and Limitations

Direct QA, definition QA, document guidance QA, exception-based QA, comparison QA, procedure QA, and scenario-based QA reach perfect end-to-end pass rates in the final report. The weakest groups are labor-dispute procedure QA, hard-negative citation QA, paraphrased real-user QA, multi-hop QA, and

table or appendix lookup. These weak categories share a common pattern: they require the system to infer a legal issue from informal wording, avoid similar but wrong provisions, or find specific appendix/table evidence.

For graph-required queries, end-to-end pass rate is 0.605. For queries not requiring graph, it is 0.912. This does not mean the graph is harmful. It means graph-required queries are harder. They often require multiple legal bases and are more likely to expose retrieval gaps.

The evaluation is reproducible but bounded. It checks citation coverage, citation containment, answer shape, and refusal behavior. It does not prove full legal correctness. The corpus contains six document groups, so the assistant cannot answer every Vietnamese labor-law question. The out-of-corpus pass rate is strong on the six refusal tests, but broader out-of-scope detection would require a larger and more varied benchmark.

The most important technical limitation is retrieval robustness. Citation validation prevents unsupported citations, but it cannot repair missing evidence. When the retriever misses a required provision, the answer can be grounded in retrieved context and still fail the benchmark because the retrieved context is incomplete.

The final results support a nuanced conclusion. Graph expansion improves retrieval over hybrid-only search and citation validation prevents benchmark-level citation hallucination. The system is effective as a scoped, citation-grounded legal information assistant. The remaining work is not primarily about making the generator more fluent. It is about making retrieval more robust for informal language, tables, hard negatives, and cross-document procedure.

6 Conclusion and Future Work

6.1 Conclusion

This thesis presented a complete AI/software system for citation-grounded Vietnamese labor-law question answering. The system combines hierarchy-aware corpus processing, Qdrant hybrid retrieval, Neo4j graph expansion, deterministic citation validation, FastAPI backend services, a Next.js frontend, and deterministic evaluation. The work is intentionally scoped: it supports legal information access over six indexed document groups and refuses unsupported questions when the indexed corpus is insufficient.

The main design lesson is that legal QA requires more than semantic similarity. Vietnamese labor-law answers must preserve hierarchy, exact statutory coordinates, cross-references, and legal rank. A flat or hybrid retriever can recover many relevant passages, but it can miss implementing regulations, appendices, or procedure provisions. Graph expansion helps by traversing legal structure around retrieved seeds, while deterministic citation validation prevents final answers from citing outside retrieved evidence.

The final 100-query benchmark shows both strengths and limitations. Graph-augmented retrieval improves Recall@10 from 0.735 to 0.835 and retrieval pass rate from 0.649 to 0.766. The final end-to-end pass rate is 0.780. Citation validation pass rate is 1.000, and out-of-corpus QA pass rate is 1.000. The main failures remain retrieval missing required context, over-expansion to distracting citations, low-information extractive answers, and one answer missing a required rule.

6.2 Future Work

Future work should first improve query understanding for informal, paraphrased user questions. Many failures occur when the user describes a practical situation without naming the legal issue. A safer query rewriting layer could map these questions to legal topics and article candidates before retrieval.

Second, table and appendix retrieval should become a first-class subsystem. Retirement schedules, permitted-work lists, and official templates are often stored as table or appendix evidence. Treating these as ordinary text chunks loses important structure. Future work should parse tables into row-level records with explicit coordinate metadata.

Third, graph expansion needs stronger hard-negative filtering. The final evaluation shows that graph expansion improves recall but can introduce related yet distracting legal bases. Future expansion should consider edge type, query intent, legal rank, and benchmark-learned negative patterns before promoting related provisions.

Fourth, labor-dispute procedure retrieval should be improved. These questions require cross-code links between labor rights and civil procedure rules. A targeted procedure module may be more reliable than broad graph expansion.

Fifth, the corpus should be expanded and versioned. A practical legal assistant must track amendments, new decrees, new circulars, validity periods, and superseded provisions. Each corpus update should trigger index rebuilds, graph rebuilds, and benchmark reruns.

Finally, evaluation should include expert review. Deterministic software checks are valuable because they are reproducible and expose system failures clearly. They cannot replace legal expert judgment about interpretation, completeness, and practical advice. The next benchmark should combine deterministic citation tests, expert-labeled answer reviews, and anonymized real user queries.

The implemented system shows that a citation-grounded legal assistant is best treated as an integrated software system. Retrieval, graph structure, validation, refusal behavior, deployment, and evaluation all matter. The language model is only one component. For legal QA, the larger engineering challenge is building the evidence pipeline around it.

References

[1] Ilias Chalkidis et al. “LexGLUE: A Benchmark Dataset for Legal Language Understanding in English”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. 2022.

[2] Darren Edge et al. “GraphRAG: Unlocking LLM Discovery on Narrative Private Data”. In: *arXiv preprint arXiv:2404.16130* (2024).

[3] Shahul Es et al. “Ragas: Automated Evaluation of Retrieval Augmented Generation”. In: *arXiv preprint arXiv:2309.15217* (2023).

[4] Neel Guha et al. “LegalBench: A Collaboratively Curated Benchmark Dataset for Measuring Legal Reasoning in Large Language Models”. In: *arXiv preprint arXiv:2308.11462* (2023).

[5] Ziwei Ji et al. “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12 (2023), pp. 1–38.

[6] Vladimir Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. 2020.

[7] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. 2020.

[8] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 2019.

[9] Jon Saad-Falcon et al. “ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems”. In: *arXiv preprint arXiv:2311.09476* (2023).

A Official Artifacts

A.1 Primary Sources Read for the Thesis

The final thesis was written from the implemented repository and official artifacts:

- `README.md`
- `docs/ARCHITECTURE.md`
- `docs/REPRODUCIBILITY.md`
- the preserved current thesis source under `thesis/original/`
- `artifacts/index/current.json`
- `artifacts/evaluation/golden_benchmark_100_extended.jsonl`
- `artifacts/evaluation/final/ablation_retrieval_final_report.md`

- `artifacts/evaluation/final/ablation_retrieval_final_results.json`
- `artifacts/evaluation/final/end_to_end_final_report.md`
- `artifacts/evaluation/final/end_to_end_final_results.json`

A.2 Artifact Availability Note

The current checkout contains the official index manifest and final evaluation outputs. The reproducibility guide references generated chunk and graph paths such as `artifacts/chunks/` and `artifacts/graph/`; those directories are not present in this checkout. The final thesis therefore uses `artifacts/index/current.json` for corpus/index counts and `artifacts/evaluation/final/` for evaluation metrics.

B Metric Formulas

B.1 Retrieval Metrics

For an in-corpus query q, let $G(q)$ be the required citation set and let $\mathcal{C}_{10}(q)$ be the citation coordinates retrieved in the top-10 context window. Recall@10 is:

$$\text{Recall}@10(q) = \frac{|G(q) \cap \mathcal{C}_{10}(q)|}{|G(q)|}.$$

The reported Recall@10 is the macro-average over 94 in-corpus queries. Required Citation Coverage is reported with the same citation-level coverage definition in the official ablation report.

For forbidden citations, let $F(q)$ be the benchmark-defined forbidden set. The violation rate is:

$$\text{ForbiddenViolationRate} = \frac{1}{94} \sum_{q} \mathbb{I}\left[F(q) \cap \mathcal{C}_{10}(q) \neq \emptyset\right].$$

B.2 Adjusted End-to-End Score

The 100-query benchmark contains 94 in-corpus queries and 6 out-of-corpus refusal tests. For in-corpus queries, a pass requires retrieval, answer, citation, and quality checks. For out-of-corpus queries, a pass requires refusal or insufficient-context behavior. The adjusted end-to-end score is:

$$\text{AdjustedE2E} = \frac{\sum_{q \in Q_{\text{in}}} \mathbb{I}[\text{Pass}_{\text{in}}(q)] + \sum_{q \in Q_{\text{ooc}}} \mathbb{I}[\text{RefusalPass}(q)]}{100}.$$

The final official score is 0.780.

C Reproducibility Commands

C.1 Retrieval Ablation

```
.venv\Scripts\python.exe scripts\ablation_retrieval_100.py `
  --benchmark-path artifacts\evaluation\golden_benchmark_100_extended.jsonl `
  --output-prefix ablation_retrieval_100_final_candidate
```

C.2 End-to-End Evaluation

```
.venv\Scripts\python.exe scripts\evaluate_end_to_end_rag.py `
  --benchmark-path artifacts\evaluation\golden_benchmark_100_extended.jsonl `
  --output-prefix end_to_end_100_final_candidate
```

D Representative Benchmark Cases

D.1 Successful In-Corpus Case

Query: *Female retirement age in 2026.*

Required evidence: Labor Code retirement-age rule and Decree 135/2020/ND-CP table evidence.

Outcome: pass. The system retrieves the relevant retirement-age evidence and returns a citation-grounded answer.

D.2 Successful Out-of-Corpus Case

Query: *Minimum wage by region in 2026.*

Expected behavior: refuse or report insufficient indexed context.

Outcome: pass. The required legal instrument is outside the indexed corpus, so the system does not provide an unsupported answer.

D.3 Remaining Failure Case

Query: *The company says there is little work and transfers me to another job for nearly three months without asking me; is that acceptable?*

Required evidence: Labor Code Article 29 clauses on temporary transfer.

Outcome: fail. The final report marks the case as a missing-required-context retrieval failure.